Report on homework 1 -Linear and Binary Search Performance (Haoran Ma)

Background

Compare the performance of two search algorithms, Linear Search and Binary Search, when applied to astronomical observation data. The primary objectives were to:

- Parse observation data containing speed and brightness values.
- Compute signatures for each observation and match them with target signature values from a separate file.
- Measure the execution time for each search method and analyze their performance.

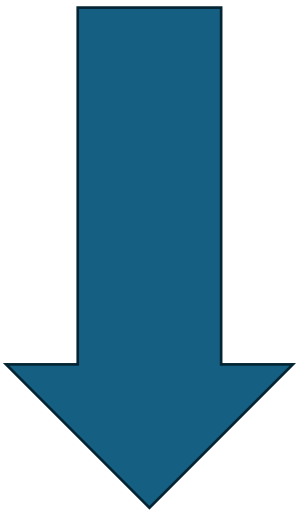
The data was processed using two key methods: Linear Search, which sequentially scans through all observations, and Binary Search, which requires pre-sorting the data and then uses a divide-and-conquer strategy for matching.

Code Analysis

Parsing and Signature Calculation

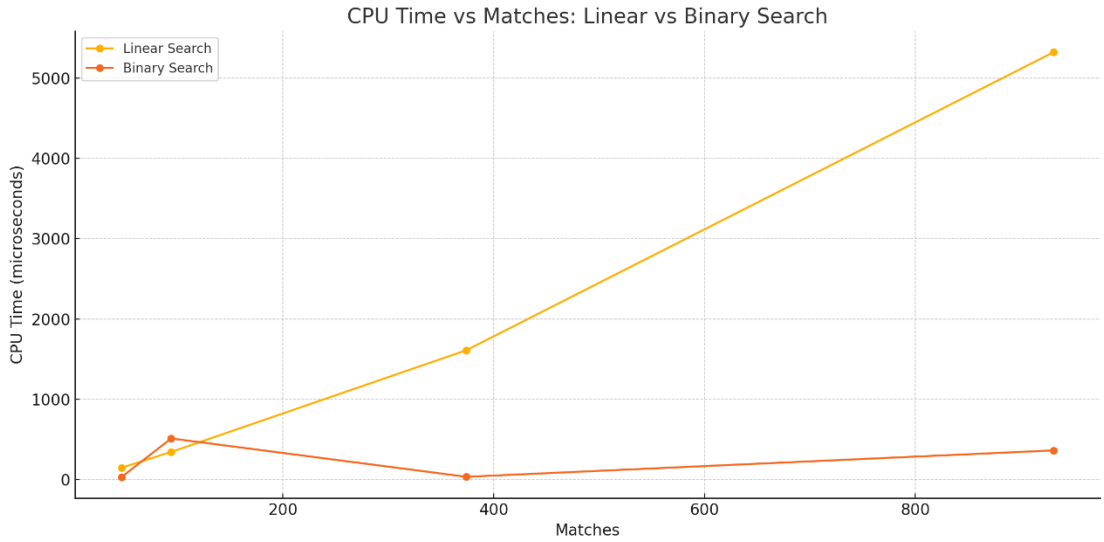
The data is parsed using a general-purpose template function, readFile, which reads lines from a file and converts them into the desired object type. For observations, the Sighting class represents speed and brightness. Each observation's signature is computed using the formula:

The signature is calculated by multiplying the speed and brightness values of the observation, dividing the result by 10, and then rounding up to the nearest integer.



More on the next page

Search Implementations Experimental Results section



Performance Measurement

Execution times were measured using the Timer class, which records elapsed time in microseconds. The time spent on sorting (for Binary Search) and the actual search process was explicitly reported.

Experimental Results

The chart provided illustrates the CPU time required for Linear and Binary Search as a function of the number of matches. Key observations include:

Linear Search Performance

1. Growth Pattern:

Linear Search has a time complexity of O(n), where n is the number of elements in the dataset. This means the runtime increases linearly as the dataset grows. This behavior is consistent across the entire range of input sizes since every element must be checked sequentially.

2. Performance at Small Scale:

For small datasets (e.g., fewer than 300 elements), Linear Search often outperforms Binary Search. This is because it avoids the O(n log n) preprocessing cost of sorting required for Binary Search. In small datasets, the overhead of sorting can dominate, making Linear Search faster for one-time searches.

3. Scaling Challenges:

As the dataset grows beyond a certain size (e.g., more than 300 elements), the linear growth in execution time becomes a significant disadvantage. The performance deteriorates quickly, making Linear Search inefficient for large datasets.

Binary Search Performance

1. Initial Overhead:

Binary Search requires the dataset to be sorted before searching. Sorting typically has a time complexity of O(n log n). For very small datasets or single-use scenarios, this overhead can make Binary Search slower than Linear Search.

2. Superior Scaling:

Once the dataset is sorted, Binary Search exhibits a time complexity of O(log n) for each search. This logarithmic growth is far more efficient than the linear growth of Linear Search. For datasets larger than approximately 400 elements, Binary Search's runtime, including sorting, becomes significantly lower than that of Linear Search.

3. Consistency:

After the dataset is sorted, Binary Search maintains stable and predictable performance. The logarithmic complexity ensures that even as the dataset size grows, the search time increases very slowly compared to Linear Search.

Conclusion part:

Analysis and Insights

The performance differences highlight the trade-offs between the two methods:

- Linear Search is straightforward and efficient for small datasets but struggles to scale efficiently due to its linear complexity.
- Binary Search, despite its upfront cost, excels in scenarios involving large datasets or repeated searches, as the sorting step is amortized across multiple queries.

The results clearly demonstrate Binary Search's advantages for large-scale applications, particularly when preprocessing time can be justified. However, for smaller datasets or one-off searches, Linear Search remains a practical and competitive choice.